

Desarrollo Web en Entorno Servidor. 2º DAW 26/27.

UT2. Ejecución de código en servidor.

UT3. Estructuras de programación web.

UT4. Aplicaciones web con estado.

UT5. Diseño de *backend* de aplicaciones.

Crea una aplicación web con varias páginas que consista en una aplicación de gestión de ventas de un restaurante de sushi por encargo. Todas las páginas de la aplicación deberán mantener el mismo aspecto (colores, fuentes, etc.), y todas contendrán un *banner* arriba con el nombre del restaurante y una declaración de copyright abajo.

La aplicación tiene que cumplir los siguientes requisitos funcionales:

1. La página de inicio mostrará un mensaje de bienvenida y un menú para acceder a las diferentes opciones disponibles: *Perfil*, *Generar pedido* y *Gestión de carta* y *Mejores clientes*. En todo momento se tiene que poder volver a la página principal.

Además, debe mostrarse un formulario de conexión donde se pueda escribir el nombre de usuario y la contraseña y conectarse a la aplicación en todas las páginas de la aplicación.

Si el usuario no se ha autenticado, no podrá acceder a ninguna de las opciones del menú excepto *Perfil*. El resto deberán estar desactivadas y además llevar a una página de error al cargarlas.

2. Para comprobar si un usuario existe, habrá que leer el archivo "usuarios.json". Este fichero será un JSON que guardará, para todos los usuarios, el nombre de usuario (*username*) y la contraseña en texto plano sin cifrar, junto a los datos personales relevantes: nombre completo, país, teléfono, correo, avatar (nombre de la imagen) y rol (cliente o administrador).

Una vez ha autenticado el usuario, en el lugar del formulario de conexión se mostrará el nombre del usuario, su avatar y un botón para desconectarse.

Cuando se use la opción de desconectar, se volverá al inicio y se destruirá toda la información asociada al usuario actual no guardada.

3. La opción *Perfil* permitirá al usuario definir sus datos, tanto la información de contacto como el avatar para la aplicación. Permite tanto editar al usuario conectado como crear y conectar un nuevo usuario.

- Se mostrará un formulario con todos los campos necesarios (aquellos que aparecen en el fichero JSON), y un botón para confirmar datos.
- El país habrá que seleccionarlo utilizando un desplegable que leerá los posibles países de un fichero CSV llamado "países.csv".
- Si ese usuario ya tuviera datos guardados, habrá que mostrar los datos guardados en el formulario.
- Se mostrará la contraseña en claro para poder cambiarla, pero no se podrá cambiar el nombre de usuario.

Habrá un avatar por defecto, y varios a escoger en el propio sistema. También habrá que dar la opción al usuario de subir uno propio que se quedará guardado de forma permanente en el servidor.

Hay que realizar varias validaciones, todas ellas a nivel de servidor. El teléfono han de ser 9 cifras empezando por 6, 7 o 9. El correo ha de tener la forma x@y.z, donde x, y z solo pueden ser letras, números, puntos o guiones bajos.

Mientras los datos no hayan sido validados, no se podrá editar/crear un nuevo usuario. Si los datos son válidos, se guardarán en el archivo "usuarios.json". **Si ese usuario ya tuviera datos guardados, habrá que sustituir los datos en el archivo por los actuales. El *username* es la identidad del usuario y no se puede modificar.**

4. La opción *Generar pedido* mostrará un formulario para crear un pedido nuevo. El pedido estará organizado en secciones. Cada estilo de sushi disponible será una sección, siendo estas *Nigiri*, *Maki* o *Temaki*. Aparte de las secciones de sushi estarán también la sección *Sashimi*, la sección *Otros platos* y la sección *Bebidas*.

La información de los distintos estilos de sushi, de los otros platos y de las bebidas disponibles, estará dentro del fichero, junto con sus precios, por pieza en el caso de sushi y sashimi, y sus alérgenos estarán en el fichero "menu.csv" adjunto, del cual no se podrá modificar el formato.

Además de los alérgenos indicados, todos los estilos de sushi y sashimi llevan sulfitos, las opciones de salmón, atún y anguila llevan pescado, y las de langostinos y cangrejo llevan crustáceos.

Habrà que mostrar las imágenes de todas las opciones y estilos y el precio de todos ellos, así como las imágenes de todos los alérgenos de cada plato. Las piezas de sushi solo se pueden pedir en parejas, y sólo se pueden pedir hasta 24 piezas de sushi del mismo tipo y sabor. El sashimi se puede pedir de manera individual, también hasta un límite de 24 piezas por sabor.

Se mostrarà un botón llamado *Comprobar ofertas*, y cada vez que se ejecute se comprobarán las ofertas disponibles en el pedido. Si el usuario selecciona 2 piezas de cada sabor de nigiris o 8 de cada sabor de makis se considerará una bandeja, y habrá que hacer un descuento del 25% para cada "Bandeja de nigiri" o "Bandeja de maki". El resto de parejas de nigiris o makis se cobrarán al precio normal. Los temakis también deberán pedirse en cantidades pares, pero no estarán incluidos en las ofertas.

El botón llamado *Comprobar ofertas* se mostrarà siempre, mientras que el botón llamado *Finalizar pedido* solo se mostrarà cuando se haya realizado al menos una vez la acción de comprobación de ofertas.

El botón llamado *Finalizar pedido* confirmará el pedido con los últimos datos disponibles, sin realizar una nueva comprobación.

Si el cliente decide finalizar el pedido, habrá que volver a comprobar que ese cliente tiene definida información de contacto en el fichero "usuarios.json". En caso afirmativo el pedido se añadirà a un fichero CSV llamado "pedidos.csv", que utilice comas de separador de campos y nueva línea para registros. Habrà que incluir el nombre del usuario, la fecha y hora actuales y la cantidad total final. En caso contrario se mostrarà un mensaje de error.

Mientras el usuario no finalice el pedido, y siempre que haya comprobado las ofertas al menos una vez, puede navegar por otras secciones de la web y volver a recuperar el pedido actual al cargar la sección *Generar pedido*, siempre que no se desconecte.

Una vez finalizado el pedido se mostrarà en pantalla una factura detallada en forma de tabla, en la que todos los importes se tienen que mostrar en formato ticket, utilizando una función de texto con formato

El ticket debe mostrar el importe con 2 decimales y el símbolo €, alineado a la derecha y rellenando con # a la izquierda de manera que la parte entera ocupe siempre 4 posiciones. A partir de este momento el usuario ya no puede recuperar el pedido.

5. La opción *Gestión de carta* solo estará disponible si el usuario conectado es el que tenga el rol "administrador", que debe existir en el fichero JSON de usuarios. Esta opción mostrará la información del archivo "menu.csv" en forma de tabla editable. Se podrán editar todos los campos de los menús ya existentes, y además añadir otros platos al menú y eliminar platos ya existentes.

Habrà un botón *Guardar* que almacenará la información en el fichero CSV del menú. Sólo se podrá guardar un plato cuando la información sea completa, en caso contrario se le indicará al usuario que falta información del plato.

6. La opción *Mejores clientes* mostrará todos aquellos clientes que hayan realizado al menos un pedido, ordenados de mayor a menor según la suma del importe de todos sus pedidos.

Para ello habrá que recorrer el fichero "pedidos.csv" y agrupar su información por nombre de usuario de cliente, para posteriormente mostrarlos de mayor a menor gasto, sumando las cantidades finales de cada pedido.

Se debe utilizar el patrón Modelo-Vista-Controlador para separar la lógica de negocio de los aspectos de presentación

Para ello, hay que diseñar una biblioteca de clases en PHP que permita generar de manera dinámica todo el código HTML de tu aplicación sin que dicho código HTML sea visible dentro del código de lógica de negocio de la aplicación.

1. Las funcionalidades de la aplicación deberán implementarse mediante clases de modelo y controlador. Las vistas se implementarán mediante plantillas PHP.

2. Existirá una clase de modelo para cada entidad: usuario, menú, y pedido. También existirá una clase madre de modelo de la que heredarán el resto de modelos.
3. Habrá que definir al menos un método en cada clase madre modelo que deberán redefinir las hijas.
4. Las clases de controladores gestionarán las entradas de los formularios, su uso correcto y la opción de visualización más adecuada.
5. En todas las clases debe usarse un mecanismo de control de errores, que tendrán que ser mostrados correctamente en la visualización.
6. **Todos los métodos deben tener sugerencias de tipos en sus parámetros de entrada y en su salida.**
7. Las vistas se definirán como plantillas que deberán ser llamadas por el controlador con los datos adecuados.
8. No se pueden utilizar controles ocultos ni cookies, pero sí sesiones.

Recomendación de diseño de arquitectura de modelos

Usuario

- leer usuarios
- buscar usuario
- comprobar credenciales
- validar datos
- crear/actualizar usuario
- guardar usuarios.

Menu

- leer menu.csv
- obtener productos
- añadir
- modificar
- eliminar
- guardar.

Pedido

- representar el pedido actual
- calcular importes
- comprobar ofertas
- guardar pedidos
- obtener histórico
- calcular mejores clientes.

Se evaluará el uso de:

- | | |
|-----------------------------|---|
| • Código insertado | • Comentarios del código |
| • Directivas | • Presentación y usabilidad |
| • Mecanismos de decisión | • Definición de clases y objetos |
| • Bucles | • Herencia de clases |
| • Matrices | • Control de errores |
| • Texto con formato | • Separación de la lógica de negocio de la presentación |
| • Funciones de usuario | • Generación dinámica del contenido web |
| • Uso de archivos | |
| • Uso de sesiones o cookies | |

Normas de entrega:

- Habrá que seguir una metodología de trabajo continuo e incremental a través del aula virtual.
- Se habilitarán varios puntos de control en el aula virtual. Solo habrá que subir de forma regular un enlace al *tag* del repositorio de GitHub que contenga la versión con el trabajo realizado hasta el momento junto a un registro de cambios.

- La versión final será entregada, incluyendo el último registro de cambios, a más tardar en la fecha señalada
- No pueden ampliarse ninguno de los plazos de entrega.

Normas de despliegue:

- La página de inicio se cargará automáticamente al entrar en la carpeta del proyecto de la unidad con el navegador.
- No se puede utilizar lenguaje JavaScript en todo el proyecto.
- Hay que aportar todo lo necesario para que el proyecto sea funcional (incluyendo hojas de estilos, recursos externos y repositorios de datos).
- Toda la información sobre cambios en la configuración del despliegue deberá ir en documento de texto plano llamado “readme.md”.
 - Solo se implementarán los cambios sugeridos cuando éstos sean necesarios.
- Si 2 proyectos cualesquiera son iguales, la nota de ambos será 0.
- Si algún proyecto no es funcional su nota será 0. Si muestra errores PHP en alguna página, su nota máxima será 5.
- ***En este proyecto se utilizará contraseña en texto plano únicamente con fines didácticos. No es un mecanismo adecuado para una aplicación real.***

Criterios de evaluación

RA 2. Escribe sentencias ejecutables por un servidor web reconociendo y aplicando procedimientos de integración del código en lenguajes de marcas.

- c) Se han utilizado **etiquetas** para la inclusión de código en el lenguaje de marcas.
- e) Se han escrito **sentencias simples** y se han comprobado sus efectos en el documento resultante.
- f) Se han utilizado **directivas** para modificar el comportamiento predeterminado.
- g) Se han utilizado los distintos **tipos de variables** y operadores disponibles en el lenguaje.
- h) Se han identificado los **ámbitos de utilización** de las variables.

RA3. Escribe bloques de sentencias embebidos en lenguajes de marcas, seleccionando y utilizando las estructuras de programación.

- a) Se han utilizado **mecanismos de decisión** en la creación de bloques de sentencias.
- b) Se han utilizado **bucles** y se ha verificado su funcionamiento.
- c) Se han utilizado matrices (**arrays**) para almacenar y recuperar conjuntos de datos.
- d) Se han creado y utilizado **funciones**.
- e) Se han utilizado **formularios web** para interactuar con el usuario del navegador web.
- f) Se han empleado **métodos para recuperar la información** introducida en el formulario.
- g) Se han añadido **comentarios** al código.

RA4. Desarrolla aplicaciones web embebidas en lenguajes de marcas analizando e incorporando funcionalidades según especificaciones.

- b) Se han utilizado **mecanismos para mantener el estado** de las aplicaciones web.
- e) Se han escrito aplicaciones que integren mecanismos de **autenticación** de usuarios.
- f) Se han utilizado **herramientas** y entornos para facilitar la programación, prueba y depuración del código.
- g) Se ha realizado **control de versiones** en el código generado.
- h) Se ha utilizado una **metodología** para la planificación, asignación y desarrollo de las funcionalidades generadas.

RA 5. Desarrolla aplicaciones web identificando y aplicando mecanismos para separar el código de presentación de la lógica de negocio.

- b) Se han analizado y utilizado mecanismos y *frameworks* que permiten realizar esta **separación** y sus características principales.
- c) Se han utilizado **objetos** y controles en el servidor para generar el aspecto visual de la aplicación web en el cliente.
- d) Se han utilizado **formularios generados de forma dinámica** para responder a los eventos de la aplicación web.
- f) Se han escrito **aplicaciones web con mantenimiento de estado y separación de la lógica de negocio**.
- g) Se han aplicado los principios y patrones de diseño de la **programación orientada a objetos**.
- h) Se ha **probado y documentado** el código.
- i) Se ha realizado **control de versiones** en el código generado.
- j) Se ha utilizado una **metodología** para la planificación, asignación y desarrollo de las funcionalidades generadas.